

Capítulo 1

Formulación de Proyectos Full Stack

Desarrollo de Software IX · Código 1493 · Primer módulo del curso

Propósito del capítulo

Guiar al estudiante desde una idea inicial hasta una propuesta técnica coherente, con objetivos, alcance, requerimientos, arquitectura, plan de desarrollo y criterios de prueba para una aplicación web Full Stack.

Tabla de contenido

1. Panorama del capítulo	3
2. Del problema a la oportunidad técnica	3
3. Objetivos del proyecto	4
4. Alcance del proyecto	5
5. Análisis de requerimientos	6
6. Diseño de la arquitectura Full Stack	7
7. Diseño de interfaz frontend	9
8. Desarrollo y pruebas	9
9. Caso guía: sistema de reservas de laboratorio	10
10. Plantilla de entrega para el estudiante	11
11. Actividades sugeridas	12
12. Rúbrica sugerida para evaluar la formulación	12
13. Cierre del capítulo	13
Anexo A. Formato resumido de propuesta	14
Anexo B. Glosario básico	14

1. Panorama del capítulo

Este capítulo traduce la primera parte del programa oficial de Desarrollo de Software IX en una guía práctica para formular proyectos Full Stack. Antes de escribir código, el estudiante debe aprender a convertir una necesidad difusa en una propuesta técnica verificable: problema, objetivos, alcance, requerimientos, arquitectura, plan de desarrollo y pruebas.

Idea central

Un proyecto Full Stack no comienza con React, Express o MongoDB. Comienza con una decisión de diseño: qué problema se resolverá, para quién, con qué límites y bajo qué evidencia se podrá decir que la solución funciona.

En el contexto del curso, formular un proyecto no significa redactar una propuesta decorativa. Significa construir el mapa que guiará el trabajo técnico durante el semestre. Una formulación débil produce repositorios confusos, APIs improvisadas, interfaces sin propósito y bases de datos que no responden a la necesidad real. Una formulación sólida permite evaluar decisiones, distribuir responsabilidades y defender técnicamente la solución.

Resultado esperado

Al finalizar este capítulo, el estudiante debe poder presentar una propuesta de proyecto Full Stack que incluya un problema claramente delimitado, objetivos evaluables, requerimientos funcionales y no funcionales, arquitectura inicial, diseño preliminar de API y base de datos, boceto de interfaz, plan de desarrollo y plan básico de pruebas.

Elemento	Pregunta que debe responder	Evidencia mínima
Problema	¿Qué situación concreta se quiere mejorar?	Descripción del contexto, usuarios afectados y consecuencias actuales.
Objetivo	¿Qué cambio observable busca el proyecto?	Objetivo general y objetivos específicos medibles.
Alcance	¿Qué entra y qué queda fuera?	Límites explícitos, entregables y supuestos.
Requerimientos	¿Qué debe hacer y bajo qué restricciones?	Lista funcional/no funcional priorizada.
Arquitectura	¿Cómo se integran frontend, backend y datos?	Diagrama o descripción de componentes y flujo.
Pruebas	¿Cómo se validará que funciona?	Casos de prueba e indicadores de aceptación.

2. Del problema a la oportunidad técnica

La primera tarea de un equipo Full Stack es comprender el problema antes de proponer una pantalla, una ruta de API o una colección en la base de datos. Muchos proyectos fallan porque confunden síntomas con problemas. Por ejemplo, “la empresa necesita una app” no es un problema; es una posible solución. Un problema mejor formulado sería: “el registro manual de solicitudes retrasa la atención, duplica datos y

dificulta conocer el estado de cada caso”.

La formulación del problema debe ubicar al lector en un contexto específico: quiénes participan, qué proceso actual existe, qué falla, qué impacto genera y por qué una solución digital puede aportar valor. La descripción debe ser suficientemente concreta para justificar decisiones posteriores de arquitectura y suficientemente limitada para que el proyecto sea viable dentro del semestre.

2.1 Señales de un buen problema

- Tiene usuarios o actores identificables, no solo una idea abstracta.
- Describe una situación actual con consecuencias observables: tiempo perdido, errores, falta de trazabilidad, baja disponibilidad, mala experiencia o alto costo operativo.
- Permite imaginar criterios de éxito: reducción de pasos, mejor control, reportes, disponibilidad, seguridad, integración o automatización.
- No presupone una tecnología desde la primera línea. La tecnología se justifica después.
- Puede ser abordado mediante una aplicación Full Stack dentro de las restricciones del curso.

2.2 Técnica de redacción

Una estructura útil para redactar el problema es: contexto + proceso actual + dificultad + consecuencia + oportunidad. Esta forma evita frases genéricas y obliga a conectar la necesidad con el diseño posterior.

Plantilla de problema

En [contexto/organización], [usuarios] actualmente realizan [proceso] mediante [método actual]. Esto provoca [dificultad observable], lo cual genera [consecuencia]. Se requiere una solución que permita [cambio esperado] mediante [capacidades generales, sin entrar aún en implementación].

Versión débil	Versión mejorada
Queremos hacer un sistema para reservas.	En laboratorios universitarios, la reserva de equipos se coordina por mensajes dispersos. Esto causa choques de horario, falta de historial y baja visibilidad de disponibilidad. Se requiere una plataforma que centralice reservas, estados y notificaciones.
La tienda necesita una página web.	Una tienda local registra pedidos por chat y hojas separadas, lo que dificulta confirmar inventario, calcular totales y dar seguimiento. Se requiere una aplicación que integre catálogo, carrito, pedidos y administración básica.
Haremos una app con IA.	El equipo académico revisa manualmente entregas extensas y no cuenta con un primer filtro de similitud, estructura o completitud. Se requiere una herramienta que apoye la revisión inicial y organice evidencias para evaluación humana.

3. Objetivos del proyecto

Los objetivos convierten el problema en una dirección de trabajo. El objetivo general expresa el cambio principal que se quiere lograr; los objetivos específicos descomponen ese cambio en resultados más concretos. En un proyecto Full Stack, los objetivos deben conectar necesidades de usuario con

componentes técnicos: interfaz, API, persistencia, seguridad e integración.

Un error común es escribir objetivos como tareas aisladas: “crear una base de datos”, “hacer un login”, “usar React”. Esas frases describen acciones técnicas, pero no explican el valor del proyecto. Un objetivo más útil indica qué se logra con la acción: “permitir que usuarios autenticados consulten el estado de sus solicitudes en tiempo real mediante una interfaz web conectada a servicios backend”.

3.1 Objetivo general

El objetivo general debe ser una oración clara que responda qué solución se construirá, para quién y con qué propósito. Puede mencionar el enfoque Full Stack, pero no debe convertirse en una lista de tecnologías.

Plantilla de objetivo general

Diseñar e implementar una aplicación web Full Stack para [usuarios/contexto] que permita [capacidad principal] con el fin de [beneficio o cambio esperado].

3.2 Objetivos específicos

Los objetivos específicos deben ser verificables. Una buena regla es redactar entre cuatro y seis objetivos: uno de análisis, uno de arquitectura, uno de backend, uno de frontend, uno de persistencia/seguridad y uno de pruebas o despliegue cuando aplique.

Tipo	Objetivo específico posible	Evidencia
Análisis	Identificar requerimientos funcionales y no funcionales del proceso seleccionado.	Matriz de requerimientos priorizada.
Arquitectura	Diseñar la estructura cliente-servidor y el flujo de datos de la solución.	Diagrama de arquitectura y explicación de componentes.
Backend	Implementar servicios REST para gestionar las entidades principales del sistema.	Rutas, controladores, validaciones y documentación API.
Frontend	Construir interfaces web que permitan ejecutar los flujos principales del usuario.	Pantallas funcionales, formularios y consumo de API.
Datos	Modelar y persistir información usando una base de datos acorde al dominio.	Modelo de datos, colecciones/tablas y operaciones CRUD.
Calidad	Validar integración, usabilidad básica y manejo de errores.	Casos de prueba, resultados y correcciones.

4. Alcance del proyecto

El alcance delimita el proyecto. Indica qué se entregará, qué funcionalidades son obligatorias, cuáles son deseables y qué elementos quedan fuera. Esta sección es crítica porque protege al equipo de prometer más de lo que puede construir y ayuda al profesor a evaluar coherencia entre propuesta, avance y entrega final.

En Desarrollo de Software IX, el alcance debe ser compatible con un proyecto de aprendizaje: suficientemente amplio para integrar frontend, backend y datos; suficientemente acotado para que el equipo pueda completarlo con calidad. Un alcance sano evita sistemas gigantescos como “un ERP completo” o “una red social total” y prefiere un flujo central completo con extensiones secundarias.

4.1 Dentro y fuera del alcance

Categoría	Dentro del alcance	Fuera del alcance
Usuarios	Roles esenciales para el flujo principal.	Roles administrativos complejos sin uso real en el MVP.
Funcionalidades	CRUD principal, autenticación si aplica, reportes básicos y validaciones.	Automatizaciones avanzadas, analítica compleja o integraciones no críticas.
Integraciones	Una integración externa bien justificada, como pasarela simulada o API pública.	Dependencias externas que bloquean el proyecto si no se obtiene acceso.
Datos	Modelo de datos necesario para la operación del MVP.	Históricos masivos, migraciones reales o inteligencia de negocio completa.
Seguridad	Cifrado de contraseñas, JWT, validación y control básico de acceso.	Auditorías completas, cumplimiento regulatorio avanzado o pentesting formal.

4.2 MVP como herramienta de alcance

El Producto Mínimo Viable o MVP es una versión mínima pero funcional de la solución. No es una maqueta incompleta ni un prototipo visual sin backend. En este curso, un MVP debe permitir que el usuario complete al menos un flujo de valor de inicio a fin: ingresar, crear o consultar datos, ejecutar una acción y observar un resultado persistido o procesado.

Regla práctica del MVP Full Stack

Si se apaga el backend o se elimina la base de datos y la aplicación sigue pareciendo funcionar, probablemente no es un MVP Full Stack real. El flujo principal debe depender de la integración entre capas.

5. Análisis de requerimientos

Los requerimientos describen lo que el sistema debe hacer y las condiciones bajo las cuales debe operar. Son el puente entre el problema y la arquitectura. Sin requerimientos, el diseño se vuelve una colección de gustos técnicos; con requerimientos claros, cada tecnología y cada componente puede justificarse.

5.1 Requerimientos funcionales

Los requerimientos funcionales especifican comportamientos del sistema. Normalmente se redactan desde la perspectiva del usuario o del sistema: registrar, consultar, actualizar, eliminar, autenticar, notificar, generar, validar, filtrar, exportar o integrar.

ID	Requerimiento funcional	Prioridad	Criterio de aceptación
RF-01	El sistema debe permitir registrar usuarios con datos mínimos y validación de correo.	Alta	No se puede crear un usuario sin correo válido ni contraseña segura.
RF-02	El usuario autenticado debe crear una solicitud con categoría, descripción y evidencia opcional.	Alta	La solicitud queda persistida y visible en su historial.
RF-03	El administrador debe cambiar el estado de una solicitud.	Media	El cambio actualiza la base de datos y se refleja en la vista del usuario.
RF-04	El sistema debe permitir filtrar solicitudes por estado y fecha.	Media	La lista responde al filtro sin recargar manualmente datos inconsistentes.

5.2 Requerimientos no funcionales

Los requerimientos no funcionales describen atributos de calidad: seguridad, rendimiento, disponibilidad, usabilidad, mantenibilidad, escalabilidad, accesibilidad, compatibilidad y trazabilidad. En proyectos académicos suelen descuidarse, pero son esenciales para demostrar criterio de ingeniería.

Categoría	Ejemplo de requerimiento no funcional	Cómo se evidencia
Seguridad	Las contraseñas deben almacenarse cifradas y las rutas privadas deben requerir token válido.	Uso de bcrypt/JWT o mecanismo equivalente; pruebas de acceso no autorizado.
Usabilidad	El usuario debe completar el flujo principal sin instrucciones externas.	Interfaz con navegación clara, mensajes de error y estados visibles.
Rendimiento	Las consultas principales deben responder en un tiempo razonable para el conjunto de datos esperado.	Pruebas con datos semilla y mediciones básicas.
Mantenibilidad	El backend debe separar rutas, controladores, modelos y servicios.	Estructura de carpetas y nombres consistentes.
Confiabilidad	El sistema debe manejar errores de validación y errores del servidor sin romper la interfaz.	Respuestas HTTP apropiadas y mensajes de UI controlados.

6. Diseño de la arquitectura Full Stack

La arquitectura describe cómo se organiza la solución. En este curso, la arquitectura mínima suele incluir una aplicación frontend, un servidor backend, una base de datos y posibles servicios externos. La arquitectura no es solo un dibujo: es una explicación de responsabilidades, comunicación, datos, seguridad y decisiones de tecnología.

Capa	Responsabilidad	Decisiones típicas
Frontend	Presentar interfaz, capturar acciones del usuario, validar datos básicos y consumir APIs.	React, componentes, rutas, manejo de estado, formularios, estilos.

Capa	Responsabilidad	Decisiones típicas
Backend	Exponer servicios, aplicar reglas de negocio, autenticar, validar, coordinar persistencia e integraciones.	Node.js, Express, REST, middlewares, controladores, JWT.
Base de datos	Persistir entidades del dominio y permitir consultas coherentes.	MongoDB, Mongoose, modelos, índices, relaciones por referencia o embebidas.
Servicios externos	Ampliar capacidades con pagos, IA, correo, mapas u otras APIs.	API keys, manejo de errores, límites, simulación si no hay acceso real.

6.1 Flujo cliente-servidor

El flujo básico inicia cuando el usuario interactúa con una pantalla. El frontend envía una solicitud HTTP al backend. El backend valida la solicitud, ejecuta reglas de negocio, consulta o modifica la base de datos y devuelve una respuesta. El frontend interpreta esa respuesta y actualiza la interfaz. Este ciclo debe estar claro desde la formulación porque afecta requerimientos, rutas, modelos y pruebas.

Descripción textual de arquitectura

Usuario → Interfaz React → Servicio HTTP del frontend → API Express → Middleware de autenticación/validación → Controlador → Modelo Mongoose → MongoDB → Respuesta JSON → Actualización de UI.

6.2 Diseño preliminar de API

Diseñar la API antes de implementarla evita improvisación. La API define recursos, rutas, métodos HTTP, entradas, salidas y errores. Para un MVP, no se necesita una documentación extensa tipo OpenAPI completa, pero sí una tabla clara de endpoints principales.

Método	Ruta	Uso	Entrada	Respuesta esperada
POST	/api/auth/login	Autenticar usuario.	correo, contraseña	token, datos básicos del usuario
GET	/api/solicitudes	Listar solicitudes visibles.	token, filtros opcionales	arreglo de solicitudes
POST	/api/solicitudes	Crear solicitud.	categoría, descripción	solicitud creada
PUT	/api/solicitudes/:id/estado	Cambiar estado.	nuevo estado	solicitud actualizada
DELETE	/api/solicitudes/:id	Eliminar o cancelar solicitud.	id	confirmación o estado final

6.3 Diseño preliminar de base de datos

El diseño de datos debe partir de entidades del dominio, no de pantallas. Una entidad representa algo que el sistema necesita recordar: usuario, solicitud, reserva, producto, pedido, pago, comentario, archivo o notificación. En MongoDB, el equipo debe decidir qué datos se embeben, qué datos se referencian y qué validaciones se aplican con Mongoose.

Entidad	Campos esenciales	Relaciones o notas
Usuario	nombre, correo, hashPassword, rol, estado	Puede crear solicitudes; puede administrar según rol.
Solicitud	titulo, descripcion, categoria, estado, prioridad, creador, fechas	Referencia al usuario creador; historial opcional.
Comentario	solicitudId, autor, texto, fecha	Puede embebirse en solicitud si el volumen es bajo.
Evidencia	solicitudId, nombreArchivo, url, tipo	Puede almacenarse como metadato si el archivo vive en otro servicio.

7. Diseño de interfaz frontend

El diseño de interfaz en este capítulo no exige una maqueta perfecta, pero sí una definición clara de pantallas y flujos. La interfaz debe reflejar las tareas del usuario, no solo la estructura de la base de datos. Cada pantalla debe tener un propósito, datos requeridos, acciones disponibles y estados de error o carga.

Pantalla	Propósito	Elementos mínimos
Inicio / dashboard	Orientar al usuario y mostrar información relevante.	Resumen, accesos principales, estado de sesión.
Listado	Consultar registros y aplicar filtros.	Tabla o tarjetas, filtros, búsqueda, paginación si aplica.
Formulario	Crear o editar información.	Campos validados, mensajes de error, acciones guardar/cancelar.
Detalle	Ver información completa y acciones contextuales.	Datos principales, historial, cambios de estado.
Autenticación	Controlar acceso a funciones privadas.	Login, registro si aplica, recuperación opcional.

Una buena práctica es construir un mapa de navegación antes de programar. Este mapa debe indicar qué rutas existen, qué rol puede acceder a cada una y qué datos consume. En React, esto se traduce en rutas, componentes, hooks y servicios de API.

8. Desarrollo y pruebas

La última unidad del módulo conecta formulación con ejecución. El equipo debe proponer cómo desarrollará y verificará la solución. Aunque el desarrollo completo ocurra en módulos posteriores, desde el inicio deben existir criterios de calidad: qué se probará, qué se considerará terminado y cómo se manejarán errores.

8.1 Plan mínimo de desarrollo

- 1 Preparar repositorio, estructura de carpetas, convenciones de ramas y archivo README.
- 2 Configurar backend con Node.js, Express, variables de entorno y conexión a base de datos.
- 3 Definir modelos principales y datos semilla.

- 4 Implementar endpoints del flujo principal con validación y manejo de errores.
- 5 Configurar frontend con React, rutas, layout base y servicios HTTP.
- 6 Conectar pantallas al backend y validar flujo completo.
- 7 Agregar seguridad mínima, documentación y pruebas de aceptación.

8.2 Pruebas que deben planificarse

Tipo de prueba	Pregunta	Ejemplo
Funcional	¿El flujo cumple el requerimiento?	Crear solicitud y verla en el listado.
Integración	¿Frontend, backend y base de datos se comunican correctamente?	Formulario React envía POST y MongoDB persiste.
Validación	¿El sistema rechaza datos inválidos?	Correo inválido o campos obligatorios vacíos.
Seguridad básica	¿Las rutas privadas bloquean usuarios no autenticados?	GET privado sin token responde 401.
Usabilidad	¿El usuario entiende qué hacer y qué ocurrió?	Mensajes de error, confirmaciones y estados de carga.

Criterio de cierre del módulo

Un proyecto está bien formulado cuando otra persona puede leer la propuesta y entender qué se construirá, por qué, para quién, qué incluye, qué no incluye, cómo se conectan sus capas y cómo se validará.

9. Caso guía: sistema de reservas de laboratorio

Para ilustrar el proceso, se presenta un caso guía breve. No es un proyecto obligatorio, sino un ejemplo de cómo aplicar los elementos del capítulo.

9.1 Problema

En una facultad universitaria, la reserva de laboratorios y equipos se coordina mediante mensajes, hojas compartidas y confirmaciones manuales. Esto provoca choques de horario, poca visibilidad de disponibilidad y dificultad para auditar quién solicitó un recurso. Se requiere una aplicación web que centralice reservas, disponibilidad y aprobación básica.

9.2 Objetivo y alcance

Objetivo general: diseñar e implementar una aplicación web Full Stack para gestionar reservas de laboratorios, permitiendo que estudiantes o docentes soliciten espacios y que un administrador apruebe, rechace o consulte solicitudes.

Incluye	No incluye en el MVP
Registro/authenticación básica de usuarios.	Integración real con cerraduras inteligentes o sensores físicos.

Incluye	No incluye en el MVP
Calendario o listado de disponibilidad.	Optimización automática de horarios.
Solicitud, aprobación y rechazo de reservas.	Facturación, pagos o reservas multi-campus complejas.
Panel administrativo y estados.	Aplicación móvil nativa.

9.3 Arquitectura inicial

Frontend React con rutas para login, dashboard, calendario/listado, formulario de reserva y panel administrativo. Backend Express con rutas REST para autenticación, laboratorios y reservas. Base de datos MongoDB con colecciones de usuarios, laboratorios y reservas. Seguridad básica con hash de contraseñas y JWT para rutas privadas.

Recurso	Endpoints mínimos
auth	POST /api/auth/register, POST /api/auth/login, GET /api/auth/me
laboratorios	GET /api/laboratorios, POST /api/laboratorios
reservas	GET /api/reservas, POST /api/reservas, PUT /api/reservas/:id/estado

10. Plantilla de entrega para el estudiante

La siguiente plantilla puede usarse como estructura mínima para la primera entrega de formulación. Cada sección debe completarse con información concreta del proyecto seleccionado.

- Título del proyecto y nombre de integrantes.
- Descripción del contexto y problema.
- Usuarios o actores principales.
- Objetivo general.
- Cuatro a seis objetivos específicos.
- Alcance: dentro, fuera, supuestos y restricciones.
- Requerimientos funcionales priorizados.
- Requerimientos no funcionales priorizados.
- Arquitectura propuesta: frontend, backend, base de datos e integraciones.
- Tabla preliminar de endpoints.
- Modelo preliminar de datos.
- Pantallas principales o mapa de navegación.
- Plan de desarrollo por hitos.
- Plan básico de pruebas y criterios de aceptación.

Recomendación de formato

La entrega debe ser clara y auditable. Evitar párrafos genéricos, capturas sin explicación o listas de tecnologías sin justificación. Cada decisión técnica debe poder conectarse con un requerimiento o una restricción.

11. Actividades sugeridas

Actividad 1: diagnóstico de problema

Seleccione un proceso real o plausible que pueda mejorarse con una aplicación web. Redacte tres versiones del problema: una versión inicial, una versión corregida y una versión final usando la plantilla de contexto, proceso, dificultad, consecuencia y oportunidad.

Actividad 2: matriz de requerimientos

Construya una matriz con al menos ocho requerimientos funcionales y cinco no funcionales. Cada requerimiento debe tener prioridad, criterio de aceptación y componente relacionado.

Actividad 3: arquitectura defendible

Dibuje o describa la arquitectura inicial. Luego justifique por qué cada capa existe, qué datos maneja y qué errores debe controlar. La explicación debe incluir al menos un flujo completo de usuario desde la interfaz hasta la base de datos.

Actividad 4: revisión entre pares

Intercambie la propuesta con otro equipo y responda: ¿el problema está claro?, ¿el alcance es viable?, ¿los requerimientos son verificables?, ¿la arquitectura conecta con los requerimientos?, ¿hay riesgos técnicos no reconocidos?

12. Rúbrica sugerida para evaluar la formulación

Criterio	Excelente	Satisfactorio	Insuficiente
Problema y contexto	Problema específico, evidencia clara y usuarios definidos.	Problema entendible pero con contexto parcial.	Problema genérico o confundido con la solución.
Objetivos y alcance	Objetivos medibles y alcance viable con exclusiones claras.	Objetivos adecuados pero con límites incompletos.	Objetivos vagos o alcance sobredimensionado.
Requerimientos	Funcionales y no funcionales priorizados con criterios de aceptación.	Lista básica con algunos criterios verificables.	Requerimientos ambiguos, incompletos o no priorizados.
Arquitectura	Capas, datos, API e integraciones coherentes y justificadas.	Arquitectura general entendible con algunas omisiones.	Tecnologías listadas sin relación clara con el problema.
Pruebas y calidad	Plan de validación conectado con requerimientos y riesgos.	Pruebas básicas del flujo principal.	No hay criterios claros para validar la solución.

13. Cierre del capítulo

La formulación de proyectos Full Stack es una práctica de ingeniería. Obliga a pensar antes de programar, a justificar tecnologías, a reconocer límites y a definir cómo se validará la solución. En los siguientes módulos, las decisiones tomadas aquí se convertirán en configuración del entorno, servicios backend, modelos de datos, interfaces React, seguridad e integración.

El estudiante que domina este primer capítulo no solo puede proponer una aplicación; puede explicar por qué esa aplicación debe existir, qué problema resuelve, cómo se estructura y cómo se sabrá que cumple su propósito.

Pregunta final de reflexión

Si otro equipo recibiera su documento de formulación, ¿podría construir el mismo MVP sin pedir explicaciones adicionales? Si la respuesta es no, la formulación todavía necesita precisión.

Anexo A. Formato resumido de propuesta

Campo	Contenido esperado
Título	Nombre breve y claro del proyecto.
Problema	Contexto, usuarios, proceso actual, dificultad y consecuencia.
Objetivo general	Solución Full Stack y beneficio principal.
Objetivos específicos	Resultados verificables de análisis, arquitectura, backend, frontend, datos y pruebas.
Alcance	Incluye, excluye, supuestos y restricciones.
Requerimientos	Funcionales y no funcionales con prioridad y criterio de aceptación.
Arquitectura	Capas, tecnologías, flujo de datos, seguridad e integraciones.
API	Tabla de endpoints mínimos.
Datos	Entidades principales y relaciones.
Interfaz	Pantallas y rutas principales.
Pruebas	Casos de aceptación para el flujo principal.

Anexo B. Glosario básico

Término	Definición operativa
Full Stack	Enfoque de desarrollo que integra interfaz, lógica del servidor y persistencia de datos.
MVP	Versión mínima funcional que permite validar el flujo central de valor.
API REST	Conjunto de endpoints HTTP para consultar o modificar recursos.
Middleware	Función intermedia que procesa solicitudes antes de llegar al controlador.
JWT	Token usado para representar una sesión o identidad en rutas protegidas.
CRUD	Operaciones de crear, leer, actualizar y eliminar información.
Requerimiento no funcional	Condición de calidad o restricción bajo la cual debe operar el sistema.